

REPORTE INFORMATIVO

Práctica Burbuja

Concepto general

Burbuja ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO).

Definición

Es un programa en C++ que implementa el algoritmo de ordenamiento de burbuja (*Bubble Sort*) utilizando Programación Orientada a Objetos (POO). El sistema gestiona datos numéricos almacenados en archivos de texto, permitiendo su lectura, procesamiento en memoria dinámica y exportación de resultados.

Objetivo

El objetivo fue aprender a usar la Programación Orientada a Objetos (POO) para manejar datos de forma más profesional. También sirvió para practicar el uso de memoria dinámica (punteros), el manejo de archivos (leer y escribir) y poner condiciones para que el programa no avance si no has hecho los pasos anteriores.

Proceso de elaboración

Para empezar a armar el programa, primero definimos la clase Burbuja con sus atributos privados, que son el arreglo de números y las variables de control que nos sirven para saber en qué estado se encuentra el proceso. Después desarrollamos el método para leer archivos, el cual es capaz de identificar cuántos datos tiene el documento para apartar exactamente el espacio necesario en la memoria. Una vez que los datos están listos, implementamos el algoritmo de la burbuja que compara los números por parejas y los va moviendo de lugar hasta que todo queda en orden. También incluimos una función para mostrar los resultados en pantalla y otra para escribir el resultado final en un archivo de texto. Finalmente, todo se unió en la función `main()`, donde creamos un objeto de nuestra clase y pusimos un menú con switch para que el usuario pueda navegar por las opciones de manera sencilla.

Errores y correcciones

El problema más grande que tenía el código al principio era que no estaba orientado a objetos, lo que hacía que todo estuviera amontonado y fuera difícil de manejar, por lo que la solución fue encapsular toda la lógica dentro de la clase Burbuja para que el programa fuera modular. Otro error importante era la falta de control en los pasos, ya que se podía intentar guardar un archivo que ni siquiera se había ordenado, así que lo corregí agregando banderas booleanas que revisan si ya cumpliste con el paso anterior antes de dejarte avanzar. Por último, el arreglo tenía un límite de números fijo que no dejaba trabajar con archivos grandes, pero lo arreglé implementando memoria dinámica con punteros para que el programa se adapte solito al tamaño de cualquier archivo.

REPORTE INFORMATIVO

Práctica Merge

Concepto general

Merge sort ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO).

Definición

Este programa es un sistema desarrollado en C++ para organizar listas de números de forma eficiente. Su función principal es tomar datos desde un archivo de texto externo, cargarlos en un arreglo dinámico y utilizar el algoritmo de ordenamiento por mezcla, mejor conocido como Merge Sort, para acomodarlos de menor a mayor. Al terminar el proceso, el programa permite generar un nuevo archivo con los datos ya ordenados.

Objetivo

El objetivo de este trabajo fue profundizar en la Programación Orientada a Objetos para crear un código más escalable y aprender a implementar algoritmos de ordenamiento más avanzados. También sirvió para practicar la recursividad, que es necesaria para que el Merge Sort funcione, y para reforzar el manejo de memoria dinámica mediante el uso de punteros, asegurando que el programa solo use el espacio necesario para cada archivo.

Proceso de elaboración

Para construir el programa, primero definimos la clase Merge con sus atributos privados, incluyendo el puntero para el arreglo y las variables booleanas que controlan el flujo del menú. Después desarrollamos el método para leer archivos, el cual escanea el documento para determinar el tamaño exacto del arreglo antes de copiar los números. El núcleo del programa se hizo mediante dos funciones: una encargada de dividir el arreglo en partes más pequeñas de forma recursiva y otra que mezcla esas partes ya ordenadas. También incluimos una interfaz básica en el main que utiliza un menú interactivo para que el usuario pueda cargar, ordenar, mostrar y guardar sus archivos siguiendo un orden lógico. Finalmente, nos aseguramos de liberar la memoria en el destructor para evitar errores en el sistema.

Errores y correcciones

El error más grande al principio era que el código no estaba orientado a objetos, lo que hacía que la lógica de la recursividad fuera muy difícil de controlar, así que lo corregí encapsulando todo dentro de la clase Merge para que las funciones pudieran compartir los mismos datos de forma ordenada. Otro problema era que el algoritmo intentaba ejecutarse sin haber cargado datos primero, lo que provocaba que el programa se cerrara inesperadamente, pero lo arreglé poniendo validaciones que revisan si el archivo ya fue leído y ordenado antes de permitir el siguiente paso. Por último, existía un fallo con el manejo de memoria porque se creaban arreglos temporales que no se borraban, lo que corregí usando el comando delete al final de cada mezcla para mantener la memoria de la computadora limpia.

REPORTE INFORMATIVO

Práctica Quick

Concepto general

Quick sort ordenar conjunto de caracteres, enteros y nuevo tipo de dato (PE y POO).

Definición

Este programa es una aplicación en C++ diseñada para organizar listas de números de manera muy rápida. Su función es cargar datos desde un archivo de texto, almacenarlos en memoria dinámica y aplicar el algoritmo de ordenamiento rápido, conocido como QuickSort, para acomodarlos de forma ascendente. Al final, el usuario tiene la opción de exportar los números ya ordenados a un nuevo archivo de texto para guardar su trabajo.

Objetivo

El objetivo de este ejercicio fue aprender a implementar uno de los algoritmos de ordenamiento más eficientes que existen y entender cómo funciona la técnica de "divide y vencerás". También sirvió para reforzar los conocimientos en Programación Orientada a Objetos, practicando cómo usar métodos privados para procesos internos del algoritmo y cómo gestionar punteros para que el programa sea capaz de manejar diferentes cantidades de datos sin desperdiciar memoria.

Proceso de elaboración

Para realizar el programa, primero se creó la clase Quick, donde definimos los atributos privados como el puntero del arreglo y las banderas que nos indican el estado del archivo. Después, programamos el método de partición, que es el corazón del algoritmo, ya que se encarga de elegir un pivote y acomodar los números menores a un lado y los mayores al otro. Luego, implementamos la función recursiva de QuickSort que repite este proceso hasta que toda la lista queda ordenada. También añadimos las funciones para leer el archivo de texto, contar los elementos y guardar el resultado final. Por último, organizamos todo en el main dentro de un ciclo do-while con un menú de opciones para que el programa sea fácil de usar y siga un orden lógico de ejecución.

Errores y correcciones

El error principal que corregimos fue que originalmente el código no estaba orientado a objetos, por lo que convertimos todas las funciones sueltas en métodos de la clase Quick para que el código fuera mucho más limpio y fácil de leer. Otro problema común era que el programa intentaba ordenar el arreglo antes de haber leído el archivo, así que lo solucionamos poniendo condiciones que revisan si las variables de control como archivoLeído son verdaderas antes de permitir cualquier otra acción. Finalmente, un error importante era que el arreglo se quedaba con un tamaño fijo que no siempre coincidía con el archivo, pero lo arreglamos usando memoria dinámica con new y delete, asegurando que el programa cree el espacio exacto para los números que se van a procesar.

REPORTE INFORMATIVO

Práctica Pila Arreglo Dato Base

Concepto general

Es una estructura de datos lineal que sigue el principio de que el último elemento en entrar es el primero en salir (LIFO), funcionando como una torre donde solo se puede trabajar con el elemento de arriba.

Definición

Este programa es una aplicación en C++ que realizamos para representar una "Pila Estática" utilizando un arreglo de tamaño fijo. Su función principal es permitir el almacenamiento de números enteros, controlando el acceso a través de un índice llamado "tope", lo que asegura que las inserciones y eliminaciones se hagan siempre por el extremo superior de la estructura.

Objetivo

Nuestro objetivo con este trabajo fue entender la lógica de las estructuras tipo LIFO y cómo gestionarlas mediante la Programación Orientada a Objetos. Con este ejercicio practicamos el control de índices para evitar el desbordamiento de datos, la validación de estados de la memoria y la implementación de un sistema de seguridad para que no se puedan ingresar valores repetidos dentro de la pila.

Proceso de elaboración

Para comenzar, definimos una constante MAX para limitar el tamaño de la pila y creamos la clase pila_arr con un arreglo y una variable tope. En el constructor inicializamos el tope en -1 para indicar que la pila comienza vacía. Después, desarrollamos el método push para insertar valores incrementando el índice, incluyendo un ciclo que revisa que el dato no exista ya. También programamos el método pop para eliminar el elemento superior simplemente restando uno al tope, y el método mos para imprimir la pila desde arriba hacia abajo. Finalmente, unimos todo en el main con un menú interactivo que permite realizar todas las operaciones de forma sencilla.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el main, lo que impedía que pudiéramos ejecutar las funciones de la clase manualmente; esto lo solucionamos con el ciclo do-while. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos correctamente porque no estábamos validando el estado del tope, por lo que tuvimos que añadir las funciones llena() y vacia() para evitar errores al procesar el arreglo. Otro fallo importante fue que el programa permitía ingresar el mismo número varias veces, así que añadimos un ciclo de búsqueda en el método push para evitar duplicados. Por último, arreglamos la forma de mostrar los datos.

REPORTE INFORMATIVO

Práctica Pila Arreglo Nuevo Dato

Concepto general

Es una estructura de datos tipo pila que almacena registros u objetos siguiendo la regla de que el último elemento en entrar es el primero en salir, usando un arreglo para guardarlos.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Pila Estática con Estructuras". Su función es permitir el almacenamiento de datos agrupados en un struct Dato dentro de un arreglo fijo, controlando que todas las operaciones de insertar y eliminar se realicen únicamente por el extremo superior (el tope) de la pila.

Objetivo

Nuestro objetivo con este trabajo fue aprender a manejar estructuras de datos personalizadas bajo la lógica LIFO (Last In, First Out). Con este ejercicio practicamos cómo controlar el acceso a la memoria a través de un índice, cómo validar que la pila no se desborde y cómo asegurar que no se ingresen valores repetidos dentro de los registros de la pila.

Proceso de elaboración

Para comenzar, definimos el struct Dato y creamos la clase pila_arr_nd con un arreglo de estos datos y una variable tope. En el constructor inicializamos el tope en -1 para indicar que la pila está vacía. Después, desarrollamos el método push para agregar nuevas estructuras incrementando el tope, asegurándonos primero de que el valor no exista ya en la pila. También programamos el método pop para remover el elemento del tope restando uno al índice y el método mos para mostrar los datos desde el último que entró hasta el primero. Finalmente, unimos todo en el main mediante un menú interactivo que permite realizar todas las funciones de la clase.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el main, lo que impedía que pudiéramos ejecutar las funciones de la clase de forma manual, así que lo solucionamos con un ciclo do-while. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos correctamente porque no estábamos accediendo al campo .valor de la estructura, por lo que tuvimos que ajustar esa sintaxis en todos los métodos. Otro fallo importante fue que el programa permitía valores duplicados, así que añadimos un ciclo de búsqueda dentro del push para validar cada dato antes de insertarlo. Por último, arreglamos un error en el método de mostrar datos, ya que al principio se imprimían en orden ascendente y lo corregimos para que se mostraran desde el tope hacia abajo, respetando la lógica de una pila.

REPORTE INFORMATIVO

Práctica Pila Puntero Dato Base

Concepto general

Es una estructura de datos tipo pila que utiliza memoria dinámica para conectar elementos mediante nodos y punteros, permitiendo que la pila crezca según sea necesario sin un tamaño fijo.

Definición

Este programa es una aplicación en C++ que realizamos para representar una "Pila Dinámica" utilizando punteros. Su función es permitir el almacenamiento de números enteros siguiendo la regla LIFO (el último en entrar es el primero en salir), donde cada nuevo dato se convierte en el nuevo "tope" de la estructura al enlazarse con el nodo anterior.

Objetivo

Nuestro objetivo con este trabajo fue aprender a gestionar una pila sin las limitaciones de un arreglo estático y practicar el uso de memoria dinámica. Con este ejercicio reforzamos cómo crear nodos en tiempo real, cómo mover el puntero del tope al eliminar elementos y cómo liberar la memoria con delete para mantener el programa eficiente.

Proceso de elaboración

Para comenzar, definimos un struct Nodo con el dato y un puntero al siguiente nodo, además de un puntero tope que siempre apunta al elemento superior. En el constructor inicializamos el tope en NULL. Después, desarrollamos el método push para crear nodos con new, conectándolos de forma que el nuevo nodo apunte al tope anterior y luego se convierta en el nuevo tope. También programamos el método pop para eliminar el nodo superior, moviendo el tope al siguiente enlace antes de borrar el nodo anterior. Finalmente, incluimos funciones para mostrar la pila y calcular su tamaño recorriendo los nodos, integrando todo en un menú interactivo en el main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el main, lo que impedía que pudiéramos ejecutar las funciones de la clase manualmente, así que lo solucionamos con el ciclo do-while. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos correctamente porque al insertar un nodo no estábamos guardando la dirección del tope anterior, lo que rompía la cadena; esto lo arreglamos haciendo que el nuevo nodo apunte al tope actual antes de actualizarlo. Otro fallo importante fue que el programa permitía valores duplicados, así que añadimos un recorrido previo con un puntero auxiliar en el push para validar que el dato no exista. Por último, arreglamos la función llena(), ya que en una pila dinámica solo se llena si se agota la memoria RAM, por lo que implementamos una prueba de asignación con nothrow.

REPORTE INFORMATIVO

Práctica Pila Puntero Nuevo Dato

Concepto general

Es una estructura de datos tipo pila que utiliza memoria dinámica y nodos para organizar información dentro de registros, permitiendo que la pila crezca libremente según la memoria disponible.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Pila Dinámica con Estructuras". Su función es permitir el almacenamiento de datos agrupados en un struct Dato siguiendo la lógica LIFO (último en entrar, primero en salir), conectando cada elemento mediante punteros para que el nuevo registro se convierta siempre en el tope de la pila.

Objetivo

Nuestro objetivo con este trabajo fue aprender a integrar estructuras de datos personalizadas con el manejo de nodos y punteros. Con este ejercicio practicamos cómo crear elementos en tiempo real, cómo enlazar una estructura completa dentro de un nodo dinámico y cómo asegurar que la cadena de datos se mantenga unida al insertar o eliminar el elemento superior.

Proceso de elaboración

Para comenzar, definimos el struct Dato y un struct Nodo que contiene dicho dato junto con un puntero al siguiente nivel de la pila. En el constructor inicializamos el puntero tope en NULL. Después, desarrollamos el método push para reservar memoria con new, validando primero que el valor no esté repetido; una vez validado, el nuevo nodo se enlaza al tope actual y luego toma su lugar. También programamos el método pop para mostrar el valor del tope, mover el puntero al siguiente nodo y liberar la memoria del nodo eliminado con delete. Finalmente, incluimos funciones para mostrar la pila completa y calcular su tamaño recorriendo los nodos, uniendo todo en un menú interactivo en el main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el main, lo que impedía que pudiéramos probar las funciones de la clase manualmente, así que lo solucionamos con el ciclo do-while. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos correctamente porque no estábamos accediendo bien al campo .valor de la estructura dentro del nodo, por lo que tuvimos que corregir esa sintaxis. Otro fallo importante fue que el programa permitía valores duplicados, así que añadimos un recorrido de validación con un puntero auxiliar antes de cada inserción.

REPORTE INFORMATIVO

Práctica Pila Librería Dato Base

Concepto general

Es una estructura de datos que utiliza las librerías estándar de C++ para gestionar elementos bajo la regla de que el último en entrar es el primero en salir (LIFO).

Definición

Este programa es una aplicación en C++ que realizamos para representar una pila utilizando la librería `<stack>` (STL). Su función es permitir el almacenamiento de números enteros de forma automática, facilitando las operaciones de insertar con `push` y eliminar con `pop` sin tener que programar toda la lógica de nodos o arreglos desde cero.

Objetivo

Nuestro objetivo con este trabajo fue aprender a utilizar las herramientas que ya vienen en el lenguaje para optimizar nuestro código. Con este ejercicio practicamos cómo encapsular un contenedor profesional de C++ dentro de nuestra propia clase para añadirle reglas extras, como la validación para que no se puedan ingresar valores repetidos dentro de la pila.

Proceso de elaboración

Para comenzar, incluimos la librería `<stack>` y definimos la clase `pila_lib` con un objeto de tipo `std::stack` como atributo privado. Después, desarrollamos el método `push`, donde usamos una pila auxiliar de tipo "copia" para recorrer los datos y verificar que el nuevo valor no exista antes de agregarlo oficialmente. También programamos el método `pop` para mostrar y quitar el elemento superior, y el método `mos` que imprime el contenido desde el tope hacia abajo usando también una copia temporal para no vaciar la pila real. Finalmente, conectamos todas las funciones a un menú interactivo en el `main` utilizando las funciones `empty()` y `size()` que ya vienen en la librería.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el `main`, lo que impedía que pudiéramos ejecutar las funciones de la clase para ver si la librería trabajaba bien; esto lo solucionamos con el ciclo `while` y el `switch`. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos correctamente porque la librería estándar permite duplicados por defecto, así que tuvimos que corregir el proceso de inserción creando una pila auxiliar para validar que el dato fuera único. Otro fallo importante fue que el método para mostrar los datos borraba toda la información al intentar imprimirla, por lo que tuvimos que arreglarlo usando una copia para proteger los datos originales. Por último, ajustamos la función `llena()`, ya que al usar una librería dinámica la pila no tiene un límite fijo como los arreglos.

REPORTE INFORMATIVO

Práctica Pila Librería Nuevo Dato

Concepto general

Es una estructura de datos que utiliza las librerías estándar de C++ para organizar registros u objetos bajo la regla de que el último en entrar es el primero en salir.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Pila de Estructuras" utilizando la librería `<stack>`. Su función es permitir el almacenamiento de datos agrupados en un struct `Dato`, aprovechando las funciones de la biblioteca estándar para insertar y eliminar elementos de forma automática, manteniendo siempre la lógica LIFO.

Objetivo

Nuestro objetivo con este trabajo fue aprender a combinar el uso de estructuras personalizadas con contenedores profesionales de la STL. Con este ejercicio practicamos cómo encapsular una herramienta avanzada dentro de nuestra propia clase para añadirle validaciones extra, como el recorrido de seguridad para evitar que se guarden registros con valores duplicados.

Proceso de elaboración

Para comenzar, definimos el struct `Dato` y creamos la clase `pila_lib_nd` que contiene un objeto `std::stack<Dato>` como atributo privado. Después, desarrollamos el método `push`, donde creamos una pila temporal para revisar cada registro y confirmar que el valor no exista antes de usar la función `push()` de la librería. También programamos el método `pop` para quitar el elemento superior y el método `mos` para imprimir la pila completa desde el tope hacia abajo usando una copia temporal para no perder los datos. Finalmente, implementamos las funciones para verificar el tamaño y el estado de la pila, uniendo todo en el menú interactivo del `main`.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el `main`, lo que impedía que pudiéramos ejecutar las funciones de la clase para probar el código, así que lo solucionamos con un ciclo y un `switch`. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos correctamente porque no estábamos accediendo bien al campo `.valor` de la estructura al usar las funciones `top()` y `pop()`, por lo que tuvimos que ajustar esa sintaxis. Otro fallo importante fue que la librería permite duplicados por defecto, así que tuvimos que corregir el proceso de inserción añadiendo una validación manual con una pila auxiliar. Por último, arreglamos el método para mostrar los datos porque inicialmente vaciaba la pila real al intentar imprimirla, por lo que tuvimos que usar una copia para proteger la información original.

REPORTE INFORMATIVO

Práctica Cola Arreglo Dato Base

Concepto general

Es una estructura de datos que sigue el principio de que el primero en entrar es el primero en salir, similar a una fila en la vida real.

Definición

Este programa es una aplicación en C++ que recrea el funcionamiento de una estructura de datos lineal tipo "Cola" (Queue), utilizando un arreglo de tamaño fijo. Sigue la lógica de que el primer dato en entrar es el primero en salir (FIFO), permitiendo gestionar una lista de números donde las inserciones se hacen por el final y las eliminaciones por el frente.

Objetivo

El objetivo de este ejercicio fue entender cómo funcionan las estructuras de datos básicas y cómo gestionarlas mediante la Programación Orientada a Objetos. Con esto practicamos el control de índices para el frente y el final de una lista, la validación de estados para no desbordar la memoria y la implementación de métodos que permiten consultar el tamaño o el estado de la cola de manera eficiente.

Proceso de elaboración

Para empezar, creamos la clase `cola_arr` definiendo un arreglo con un tamaño máximo de 5 espacios y dos variables enteras para rastrear dónde empieza y dónde termina la cola. En el constructor inicializamos estos índices para indicar que la cola empieza vacía. Después, desarrollamos el método para agregar elementos, el cual incluye una búsqueda para evitar que se repitan valores, y el método para eliminar, que simplemente desplaza el índice del frente. También programamos funciones auxiliares para revisar si la cola está llena o vacía y para calcular su tamaño actual restando los índices. Al final, todo se conectó en el `main` mediante un menú interactivo que le permite al usuario realizar todas las operaciones y ver los mensajes de confirmación en la consola.

Errores y correcciones

El error principal que corregimos fue que originalmente el programa no tenía un menú interactivo, lo que hacía imposible que el usuario controlara las acciones de la cola. También faltaba desarrollar las funciones para agregar, eliminar y mostrar los datos, por lo que el código no hacía nada útil; esto lo arreglé programando los métodos de inserción y borrado con sus respectivos mensajes en pantalla. Otro fallo importante era que se permitía intentar eliminar datos de una cola que ya estaba vacía o agregar a una que estaba llena, lo que causaba errores de lógica, así que puse condiciones que verifican el estado de la cola antes de hacer cualquier movimiento.

REPORTE INFORMATIVO

Práctica Cola Arreglo Nuevo Dato

Concepto general

Es una estructura de datos que organiza elementos siguiendo el orden de llegada, donde el primero en entrar es el primero en salir, usando un arreglo para guardarlos.

Definición

Este programa es una aplicación en C++ que realizamos para representar una "Cola" (Queue) utilizando un arreglo de tamaño fijo. Su función es permitir el almacenamiento de datos dentro de una estructura tipo struct, controlando que los elementos se inserten por el final y se eliminen por el frente, respetando siempre el turno de entrada.

Objetivo

Nuestro objetivo con este trabajo fue aprender a manipular estructuras de datos más complejas mediante el uso de objetos y structs. Con este ejercicio practicamos cómo gestionar los índices de una fila, validar cuando el espacio está lleno para no causar errores y asegurar que el programa pueda informar correctamente el tamaño o el estado de la cola en cualquier momento.

Proceso de elaboración

Para comenzar, definimos un struct llamado Dato para envolver los valores numéricos y luego creamos la clase cola_arr_ndcon un arreglo limitado a 5 espacios. En el constructor inicializamos los indicadores de frente y final para que la cola empezara vacía. Después, desarrollamos el método para agregar elementos, el cual revisa que el valor no esté repetido antes de guardarlo, y el método para eliminar que recorre el frente de la fila. También programamos funciones para checar si la cola está llena o vacía y para medir su tamaño actual. Finalmente, unimos todo en la función principal con un menú de opciones que permite al usuario interactuar con la cola de manera sencilla.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú para interactuar con el código, lo que impedía que pudiéramos probar si las funciones servían, así que lo solucionamos integrando un switch en el main. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar los datos porque los métodos no estaban bien vinculados con la estructura del objeto, por lo que tuvimos que reprogramar los métodos enq, deq y mos para que trabajaran correctamente con los atributos de la clase. Otro fallo importante era que el programa permitía ingresar valores duplicados, así que añadimos un ciclo que escanea la cola antes de aceptar un nuevo dato. Por último, arreglamos las condiciones de "cola llena" y "cola vacía" porque antes dejaban que el programa intentara procesar espacios inexistentes en el arreglo.

REPORTE INFORMATIVO

Práctica Cola Puntero Dato Base

Concepto general

Es una estructura de datos tipo cola que utiliza nodos y punteros para enlazarse, lo que permite que crezca de forma dinámica sin un límite de tamaño fijo.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Cola Dinámica" mediante el uso de punteros. A diferencia de las colas con arreglos, esta versión utiliza una estructura de nodos conectados entre sí, donde cada elemento sabe quién es el siguiente, permitiendo agregar tantos valores como la memoria lo permita.

Objetivo

Nuestro objetivo con este trabajo fue entender el funcionamiento de las listas enlazadas aplicadas a una cola y practicar el manejo de memoria dinámica. Con este ejercicio aprendimos a conectar nodos usando punteros para el frente y el final, y a utilizar los operadores new y delete para crear o eliminar elementos de la fila de forma eficiente sin desperdiciar espacio.

Proceso de elaboración

Para empezar, definimos un struct Nodo dentro de la clase que contiene el dato y un puntero hacia el siguiente nodo. En el constructor inicializamos los punteros frente y final en NULL para indicar que la cola está vacía. Después, desarrollamos el método enq para crear nuevos nodos y conectarlos al final de la fila, asegurándonos de que no hubiera valores repetidos. También programamos el método deq para eliminar el primer nodo, moviendo el frente al siguiente elemento y liberando la memoria del nodo eliminado. Finalmente, implementamos funciones para mostrar la cola y obtener su tamaño actual, integrando todo en un menú interactivo dentro del main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú interactivo, por lo que no podíamos probar las funciones de agregar o eliminar datos en tiempo real; esto lo solucionamos con el ciclo do-while en el main. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar datos correctamente porque al principio no estábamos actualizando el puntero final cuando la cola quedaba vacía, lo que causaba que el programa fallara. Otro fallo importante fue que no estábamos liberando la memoria con delete al eliminar un elemento, por lo que tuvimos que corregir el método deq para evitar fugas de memoria. Por último, añadimos una validación con un ciclo while para evitar que se ingresen valores repetidos, ya que originalmente el programa permitía duplicados.

REPORTE INFORMATIVO

Práctica Cola Puntero Nuevo Dato

Concepto general

Es una cola dinámica que organiza la información dentro de estructuras llamadas nodos, los cuales se enlazan mediante punteros para poder crecer sin un límite fijo de memoria.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Cola Dinámica con Estructuras". Su función es permitir el almacenamiento de datos que están envueltos en un struct Dato, utilizando nodos conectados por punteros que se crean y eliminan en tiempo real según el usuario lo necesite.

Objetivo

Nuestro objetivo con este trabajo fue aprender a combinar el uso de estructuras de datos (struct) con la memoria dinámica (punteros). Con este ejercicio practicamos cómo insertar elementos por el final de la fila y eliminarlos por el frente utilizando nodos, asegurando que el programa sea flexible y no dependa de un tamaño máximo predefinido como sucede con los arreglos.

Proceso de elaboración

Para comenzar, definimos un struct Dato para el valor numérico y un struct Nodo que contiene tanto el dato como el puntero hacia el siguiente elemento. En el constructor inicializamos los punteros frente y final en NULL. Después, desarrollamos el método enq para reservar memoria con new cada vez que se agrega un dato, verificando primero que el valor no esté repetido. También programamos el método deq para eliminar el nodo del frente y liberar la memoria con delete, actualizando la posición del siguiente elemento. Al final, implementamos funciones para mostrar los datos en pantalla y consultar el tamaño de la cola, uniendo todo en un menú interactivo en el main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el main, lo que impedía que pudiéramos ejecutar las funciones de la clase, así que lo solucionamos con un ciclo do-while y un switch. También nos dimos cuenta de que no se podían agregar ni eliminar datos correctamente porque no estábamos pasando el struct Dato de forma adecuada a los nodos, por lo que tuvimos que corregir la lógica del método enq. Otro fallo importante era que el programa permitía valores duplicados, así que añadimos un ciclo while que recorre todos los nodos para avisar si el valor ya existe antes de crearlo. Por último, arreglamos un error de memoria donde el puntero final se quedaba apuntando a una dirección inválida cuando la cola se vaciaba, asegurándonos de regresarlo a NULL en el método deq.

REPORTE INFORMATIVO

Práctica Cola Librería Dato Base

Concepto general

Es una estructura de datos tipo cola que se implementa utilizando librerías estándar de C++, lo que facilita el manejo de los elementos mediante funciones ya optimizadas.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una cola utilizando la librería `<queue>` de la Standard Template Library (STL). Su función es aprovechar las herramientas que ya vienen en el lenguaje para insertar elementos con `push` y eliminarlos con `pop`, pero manteniendo un control personalizado sobre el tamaño máximo y evitando datos duplicados.

Objetivo

Nuestro objetivo con este trabajo fue aprender a utilizar librerías estándar en lugar de programar toda la lógica desde cero, comparando cómo cambia la eficiencia del código. Con este ejercicio practicamos cómo encapsular una herramienta de la librería estándar dentro de una clase propia para añadirle validaciones extras, como un límite de capacidad (MAX) y la restricción de que no se repitan valores.

Proceso de elaboración

Para comenzar, incluimos la librería `<queue>` y definimos nuestra clase `cola_lib` con un objeto de tipo `std::queue` como atributo privado. Después, desarrollamos el método `enq`, donde utilizamos una cola temporal para recorrer los datos y verificar que el nuevo valor no exista antes de usar `push`. También programamos el método `deq` para mostrar y eliminar el elemento al frente de la fila, y el método `mos` que muestra el contenido de la cola sin borrar los datos originales. Finalmente, utilizamos las funciones `empty()` y `size()` de la librería para crear los métodos que revisan si la cola está llena o vacía, conectando todo a un menú en el `main`.

Errores y correcciones

El error principal que corregimos fue que originalmente no incluimos un menú interactivo en el `main`, lo que nos impedía manipular la cola para ver si los métodos de la librería funcionaban bien. También nos dimos cuenta de que no se podían agregar ni eliminar datos correctamente porque no estábamos controlando el límite de capacidad manual, por lo que tuvimos que añadir la validación de `lleno()` basándonos en una constante MAX. Otro fallo importante fue que la librería estándar permite duplicados por defecto, así que tuvimos que corregir el proceso de inserción creando una cola auxiliar para validar los datos antes de guardarlos. Por último, arreglamos el método de mostrar datos porque al principio borraba la cola al intentar imprimirla, así que lo corregimos.

REPORTE INFORMATIVO

Práctica Cola Librería Nuevo Dato

Concepto general

Es una cola que utiliza las librerías estándar de C++ para almacenar información organizada en estructuras, permitiendo un manejo de datos más profesional y eficiente.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una cola de estructuras (struct Dato) utilizando la librería <queue>. Su función es administrar una fila de elementos donde cada dato se inserta al final y se elimina por el frente, aprovechando las funciones optimizadas de la biblioteca estándar de C++.

Objetivo

Nuestro objetivo con este trabajo fue aprender a integrar el uso de structs con herramientas avanzadas como la STL (Standard Template Library). Con este ejercicio practicamos cómo encapsular una cola profesional dentro de nuestra propia clase para añadirle reglas personalizadas, como un límite de capacidad máxima y la validación de que no existan valores duplicados dentro de la fila.

Proceso de elaboración

Para comenzar, definimos la estructura Dato y creamos la clase cola_lib_nd que contiene un objeto std::queue<Dato> como atributo privado. Después, desarrollamos el método enq donde usamos una cola temporal para revisar los valores existentes antes de agregar uno nuevo con la función push. También programamos los métodos deq para sacar el primer elemento y mos para imprimir la cola completa sin destruirla, usando nuevamente una copia temporal. Finalmente, implementamos las funciones para verificar si la cola está llena o vacía basándonos en el tamaño definido por MAX, conectando todo al menú interactivo del main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú interactivo, por lo que no podíamos comprobar si la librería estaba guardando bien los datos; esto lo arreglamos con el ciclo y el switch en el main. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar datos correctamente porque no estábamos accediendo bien al campo. valor de la estructura dentro de la cola, así que tuvimos que corregir esa sintaxis en todos los métodos. Otro fallo importante fue que la librería acepta duplicados automáticamente, por lo que añadimos una validación manual con un bucle while para buscar coincidencias antes de insertar. Por último, arreglamos el método de mostrar datos porque inicialmente vaciaba la cola principal al intentar leerla, así que lo corregimos usando una copia para proteger la información.

REPORTE INFORMATIVO

Práctica Lista Arreglo Dato Base

Concepto general

Es una estructura de datos lineal que almacena una colección de elementos de forma contigua en la memoria, permitiendo insertar y eliminar valores en cualquier posición de un arreglo.

Definición

Este programa es una aplicación en C++ que realizamos para representar una "Lista Estática" basada en arreglos. Su función principal es gestionar un conjunto de números enteros con un límite de capacidad definido, permitiendo al usuario agregar elementos al final y buscar valores específicos para eliminarlos, recorriendo los demás datos para mantener la lista unida.

Objetivo

Nuestro objetivo con este trabajo fue aprender a manipular arreglos de manera dinámica dentro de una clase y practicar la lógica de desplazamiento de elementos. Con este ejercicio reforzamos cómo buscar un valor dentro de un índice y cómo "recorrer" los espacios del arreglo hacia la izquierda cuando se borra un dato para que no queden huecos vacíos en la estructura.

Proceso de elaboración

Para comenzar, definimos una constante MAX para limitar el tamaño del arreglo y creamos la clase ListaArr con una variable tam para llevar la cuenta de los elementos reales. En el constructor inicializamos el tamaño en cero. Después, desarrollamos el método ins para colocar nuevos valores en la siguiente posición disponible y el método eli, el cual busca un número específico; si lo encuentra, usa un ciclo para mover todos los números siguientes una posición atrás y resta uno al contador. También programamos funciones para mostrar la lista completa y consultar si está llena o vacía. Finalmente, unimos todo en el main con un menú que facilita la interacción con la lista.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú interactivo en el main, por lo que el programa no permitía realizar acciones de forma manual; esto lo solucionamos con el ciclo do-while. También nos dimos cuenta de que no se podían agregar, eliminar ni mostrar datos correctamente porque no estábamos controlando los límites del arreglo, lo que podía causar que el programa intentara escribir fuera de la memoria permitida. Otro fallo importante fue en el método de eliminación, ya que al principio solo borrábamos el número pero dejábamos el espacio vacío, por lo que tuvimos que corregirlo implementando un segundo ciclo for para desplazar los elementos y mantener la lista compacta. Por último, arreglamos las validaciones de lleno() y vacio() para que el menú avisara correctamente antes de intentar realizar una operación inválida.

REPORTE INFORMATIVO

Práctica Lista Arreglo Nuevo Dato

Concepto general

Es una estructura de datos lineal que almacena información organizada en registros o estructuras dentro de un arreglo de tamaño fijo, permitiendo manipular los datos de forma contigua.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Lista Estática con Estructuras". Su función es administrar un conjunto de datos agrupados en un struct Dato, permitiendo insertar nuevos elementos y buscar valores específicos para eliminarlos, manteniendo siempre el orden del arreglo.

Objetivo

Nuestro objetivo con este trabajo fue aprender a combinar el uso de arreglos con estructuras personalizadas dentro de una clase. Con este ejercicio practicamos la lógica de búsqueda para evitar valores duplicados y reforzamos la técnica de desplazamiento de elementos hacia la izquierda al momento de borrar un registro, asegurando que la lista no tenga espacios vacíos.

Proceso de elaboración

Para comenzar, definimos el struct Dato y creamos la clase ListaArrNd con un arreglo de objetos y un contador de tamaño. En el constructor inicializamos el tamaño en cero para indicar que la lista comienza vacía. Después, desarrollamos el método ins, que antes de guardar el dato recorre la lista para verificar que no esté repetido. También programamos el método eli, el cual localiza el valor deseado y usa un ciclo for para recorrer los elementos restantes una posición atrás. Finalmente, incluimos funciones para mostrar la lista y verificar su estado, uniendo todo en un menú interactivo dentro del main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú interactivo en el main, lo que impedía que pudiéramos agregar o eliminar datos de forma manual para probar el código. También nos dimos cuenta de que no se podían manipular los datos correctamente porque no estábamos accediendo al campo .valor de la estructura, por lo que tuvimos que corregir esa parte en las comparaciones. Otro fallo importante fue que el programa permitía ingresar el mismo número varias veces, así que añadimos un ciclo de validación en el método de inserción para evitar duplicados. Por último, arreglamos un error en la eliminación donde el contador tam no se actualizaba correctamente, lo que causaba que se mostrara información basura al final de la lista.

REPORTE INFORMATIVO

Práctica Lista Puntero Dato Base

Concepto general

Es una estructura de datos lineal que utiliza memoria dinámica para conectar elementos mediante nodos y punteros, permitiendo que la lista crezca o se reduzca de forma flexible.

Definición

Este programa es una aplicación en C++ que realizamos para representar una "Lista Simplemente Enlazada" usando punteros. Su función es permitir el almacenamiento de números enteros en nodos que se van conectando uno tras otro, lo que facilita insertar o eliminar elementos en cualquier momento sin las limitaciones de tamaño de un arreglo convencional.

Objetivo

Nuestro objetivo con este trabajo fue dominar el uso de la memoria dinámica y entender cómo se enlazan los objetos a través de direcciones de memoria. Con este ejercicio practicamos cómo recorrer una lista utilizando un puntero auxiliar, cómo reconectar los nodos cuando se borra un elemento intermedio para no romper la cadena y cómo liberar la memoria correctamente para evitar fugas.

Proceso de elaboración

Para comenzar, definimos un struct Nodo que contiene el valor entero y un puntero al siguiente nodo, junto con un puntero cabeza que marca el inicio de la lista. En el constructor inicializamos la cabeza en NULL. Después, desarrollamos el método ins para agregar valores al final de la lista, incluyendo una validación para que no se repitan datos. También programamos el método eli, el cual busca el valor deseado y ajusta los punteros de los nodos vecinos para "saltarse" el nodo que se va a borrar antes de usar delete. Finalmente, incluimos funciones para mostrar la lista y calcular su tamaño de forma dinámica, integrando todo en un menú interactivo en el main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú interactivo en el main, lo que impedía que pudiéramos probar las funciones de inserción y borrado de forma manual; esto lo solucionamos con el ciclo do-while. También nos dimos cuenta de que no se podían agregar ni eliminar datos correctamente porque al borrar un nodo se perdía la conexión con el resto de la lista, por lo que tuvimos que corregir la lógica para que el puntero del nodo anterior se conecte con el siguiente antes de liberar la memoria. Otro fallo importante fue que el programa permitía duplicados, así que añadimos un recorrido previo en el método ins para avisar si el valor ya existía. Por último, arreglamos un error donde el programa fallaba al intentar eliminar el primer elemento de la lista, asegurándonos de actualizar correctamente el puntero cabeza.

REPORTE INFORMATIVO

Práctica Lista Puntero Nuevo Dato

Concepto general

Es una estructura de datos lineal que utiliza nodos enlazados dinámicamente para almacenar información organizada en registros, permitiendo que la lista crezca o se reduzca de forma flexible.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una "Lista Dinámica con Estructuras". Su función es administrar una serie de datos guardados en un struct Dato, conectándolos a través de nodos y punteros para que la información se pueda insertar y eliminar en cualquier momento, respetando un límite máximo de elementos definido por nosotros.

Objetivo

Nuestro objetivo con este trabajo fue aprender a integrar estructuras de datos personalizadas con el manejo de memoria dinámica. Con este ejercicio practicamos cómo navegar a través de una cadena de nodos para buscar valores específicos, cómo evitar que se ingresen registros duplicados y cómo desconectar y liberar la memoria de un nodo sin perder el acceso al resto de la lista.

Proceso de elaboración

Para comenzar, definimos la estructura Dato para los valores y el struct Nodo que contiene tanto el dato como el puntero hacia el siguiente elemento. En el constructor inicializamos la cabeza en NULL y el contador de tamaño en cero. Después, desarrollamos el método ins, que utiliza un puntero auxiliar para recorrer la lista y verificar que el valor no exista antes de crear un nuevo nodo al final. También programamos el método eli, el cual busca el registro deseado y ajusta los enlaces de los nodos vecinos para poder borrarlo con delete. Finalmente, incluimos las funciones para mostrar la lista y verificar su estado, integrando todo en un menú interactivo en el main.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el main, lo que impedía que pudiéramos ejecutar las acciones de la clase de forma manual, así que lo solucionamos con un ciclo do-while. También nos dimos cuenta de que no se podían agregar ni eliminar los datos correctamente porque no estábamos accediendo de forma adecuada al campo .valor de la estructura dentro de los nodos, por lo que tuvimos que ajustar esa sintaxis en las comparaciones. Otro fallo importante fue que el programa permitía valores duplicados, así que añadimos una validación que recorre toda la lista antes de aceptar un nuevo registro. Por último, arreglamos un error de memoria donde el programa fallaba al intentar borrar el primer elemento, asegurándonos de que el puntero cabeza se mueva al siguiente nodo antes de eliminar el original.

REPORTE INFORMATIVO

Práctica Lista Librería Dato Base

Concepto general

Es una estructura de datos lineal que utiliza las librerías estándar de C++ para gestionar una lista de elementos que puede crecer o reducirse, facilitando las operaciones mediante funciones predefinidas.

Definición

Este programa es una aplicación en C++ que realizamos para administrar una lista de números enteros utilizando la librería `<list>` (STL). Su función es permitir el manejo de datos de forma sencilla, aprovechando herramientas del lenguaje para insertar elementos al final con `push_back` y eliminarlos automáticamente con `remove`, pero limitando la capacidad a un máximo de 5 espacios.

Objetivo

Nuestro objetivo con este trabajo fue aprender a utilizar contenedores profesionales de C++ para simplificar el código. Con este ejercicio practicamos cómo envolver una lista de la biblioteca estándar dentro de nuestra propia clase para añadirle reglas específicas, como el control de tamaño máximo (MAX) y la validación para que no se puedan ingresar valores que ya existan en la lista.

Proceso de elaboración

Para comenzar, incluimos la librería `<list>` y definimos la clase `ListaLib` con un objeto de tipo `std::list` privado. Después, desarrollamos el método `ins`, donde usamos un ciclo para recorrer la lista y verificar que el número no esté repetido antes de agregarlo. También programamos el método `eli`, que utiliza la función interna `remove()` y compara el tamaño de la lista antes y después de la operación para confirmar si el valor realmente se borró. Finalmente, incluimos las funciones de apoyo para mostrar los datos y consultar el tamaño actual, uniendo todo en el menú interactivo del `main`.

Errores y correcciones

El error principal que corregimos fue que originalmente no teníamos un menú en el `main`, lo que impedía que pudiéramos ejecutar las funciones de la lista de forma manual, así que lo solucionamos con el ciclo `do-while`. También nos dimos cuenta de que no se podían agregar ni eliminar datos correctamente porque no estábamos controlando el límite de capacidad, por lo que tuvimos que añadir la validación de `lleno()` comparando el tamaño de la lista con nuestra constante `MAX`. Otro fallo importante fue que la librería `<list>` permite valores duplicados por defecto, así que tuvimos que corregir la inserción agregando un bucle que revisa cada elemento antes de aceptar uno nuevo. Por último, arreglamos el mensaje de confirmación al eliminar, ya que originalmente el programa decía que se había borrado el dato aunque este no existiera en la lista.

REPORTE INFORMATIVO

Práctica Lista Librería Nuevo Dato

Concepto general

Es una estructura de datos lineal que utiliza la biblioteca estándar de C++ para organizar información dentro de registros, permitiendo un manejo dinámico y eficiente de los datos.

Definición

Este programa es una aplicación en C++ que realizamos para gestionar una lista de estructuras (struct Dato) utilizando la librería <list> de la STL. Su función es permitir el almacenamiento de valores de forma organizada, aprovechando las funciones automáticas de la biblioteca para insertar y borrar elementos, pero manteniendo un control personalizado sobre el tamaño máximo de la lista.

Objetivo

Nuestro objetivo con este trabajo fue aprender a combinar el uso de estructuras personalizadas con contenedores profesionales de C++. Con este ejercicio practicamos cómo utilizar iteradores para recorrer y borrar elementos de una lista dinámica y cómo encapsular estas herramientas dentro de nuestra propia clase para añadir validaciones de seguridad, como evitar que se repitan valores.

Proceso de elaboración

Para comenzar, definimos el struct Dato y creamos la clase ListaLibNd que contiene una std::list de tipo Dato. En los métodos de la clase, desarrollamos la función ins para agregar registros al final de la lista, usando un ciclo para confirmar que el valor no exista ya. También programamos el método eli, el cual utiliza iteradores para buscar el valor específico dentro de las estructuras y removerlo con la función erase(). Finalmente, implementamos las funciones para mostrar los datos en pantalla y consultar el tamaño, integrando todo en un menú interactivo con un switch dentro del main.

Errores y correcciones

El error principal que corregimos fue que originalmente no incluimos un menú interactivo en el main, lo que impedía que pudiéramos ejecutar las funciones de la lista de forma manual, así que lo solucionamos con el ciclo do-while. También nos dimos cuenta de que no se podían agregar ni eliminar datos correctamente porque no estábamos accediendo bien al campo .valor de la estructura al usar los iteradores de la librería, por lo que tuvimos que ajustar esa sintaxis. Otro fallo importante fue que la lista permitía duplicados por defecto, así que añadimos un bucle de validación antes de cada inserción. Por último, arreglamos el método de eliminación, ya que al principio el iterador se perdía al borrar un elemento, y lo corregimos asegurándonos de que se actualizara correctamente después de usar erase.